SOFTWARE REQUIREMENTS

SPECIFICATION

For

ARES – AI RESILIENCE AND EVALUATION SECURITY SYSTEM

07 October 2026

Prepared by

Specialization	SAP ID	Name
B.Tech CSE DS	500119040	Dhruv Aggarwal
B.Tech CSE CSF	500120324	Aikagra Gupta
B.Tech CSE CSF	500121953	Neo Gonsalves
B.Tech CSE CSF	500119525	Vansh Malik

School Of Computer Science

UNIVERSITY OF PETROLEUM & ENERGY STUDIES,

DEHRADUN-248007, Uttarakhand

Contents

1 INTRODUCTION **2**

1.1 Purpose of the Project 2
1.2 Target Beneficiary 2
1.3 Project Scope 2
1.4 References 2

2 PROJECT DESCRIPTION **3**

2.1 Reference Algorithm 3
2.2 Data / Data Structure 3
2.3 SWOT Analysis 3
2.4 Project Features 4
2.5 User Classes and Characteristics 4
2.6 Design and Implementation Constraints 4
2.7 Design Diagrams 5
2.8 Assumption and Dependencies 9

3 SYSTEM REQUIREMENTS **9**

3.1 User Interface 9
3.2 Software Interface 9
3.3 Database Interface 9
3.4 Protocols 10

4 NON-FUNCTIONAL REQUIREMENTS **10**

4.1 Performance Requirements 10
4.2 Security Requirements 10
4.3 Software Quality Attributes 10

5 OTHER REQUIREMENTS **11**

APPENDIX A: GLOSSARY **12**

APPENDIX B: ANALYSIS MODEL **12**

APPENDIX C: ISSUES LIST **12**

1 INTRODUCTION

1.1 Purpose of the Project

The rapid adoption of Large Language Models (LLMs) has introduced a new paradigm of security vulnerabilities, including adversarial prompt injections, jailbreaks, role-play hijacks, context smuggling, delimiter confusion, and data exfiltration. Conventional security systems fall short in mitigating these nuanced, context-dependent threats. The primary purpose of the ARES (AI Resilience and Evaluation Security) System is to provide an enterprise-grade AI security platform for adaptive red-teaming, closed-loop prompt hardening, and runtime enforcement. ARES aims to systematically interrogate, evaluate, and harden LLM applications against these adversarial attacks using a robust architecture that integrates multi-provider LLM judging and machine learning vector classification.

1.2 Target Beneficiary

The primary beneficiaries of the ARES system are Security Analysts, AI System Administrators, and Prompt Engineers who are responsible for ensuring the robustness of LLM applications. They use ARES to perform comprehensive red-team sweeps, continuously monitor real-time threat telemetry via the dashboard, and automatically harden vulnerable prompts. By automating the evaluation and remediation of prompt vulnerabilities, ARES significantly reduces the operational overhead of securing generative AI applications and provides actionable intelligence for incident response.

1.3 Project Scope

The implemented functional scope of ARES encompasses a comprehensive suite of security evaluation and hardening tools. Key capabilities include a Live RAG Vector Retrieval system leveraging a Qdrant Cloud vector database populated with 3,695 curated adversarial attack vectors across five exploit categories. The system features a Continuous LLM Judge that evaluates model responses and assigns severity ratings, as well as a Closed-Loop Adaptive Prompt Hardening module that extracts trace data to inject counter-defense rules. Furthermore, the system includes a Machine Learning Vector Classifier with auto-retraining capabilities, achieving high cross-validation accuracy, and generates structured security audit reports mapped to OWASP and MITRE frameworks. Future functionality may include the implementation of OIDC authentication, Role-Based Access Control (RBAC), and multi-tenant telemetry isolation, which are currently scoped out for the initial implementation phase.

1.4 References

1. ARES Project Repository: NeoGonsalves/ARES-AI-Resilience-and-Evaluation-Security-System

2. OWASP Top 10 for LLM Applications (2025)
3. MITRE ATLAS Framework
4. JailbreakBench & HarmBench Exploit Vectors
5. Qdrant Cloud Vector Database Documentation

2 PROJECT DESCRIPTION

2.1 Reference Algorithm

The ARES system utilizes several key algorithms and models to perform its security evaluations. The core ML Vector Classifier employs a Scikit-Learn linear classifier trained on 768-dimensional Gemini vector embeddings to categorize attack types. The Live RAG (Retrieval-Augmented Generation) Vector Retrieval algorithm searches the Qdrant Cloud database using cosine similarity to identify relevant historical attack vectors. The LLM Judge Pipeline utilizes state-of-the-art models (Groq Llama 3.3 70B, Google Gemini 2.5/1.5, or NVIDIA NIM Nemotron-4 340B) to heuristically evaluate target model responses for vulnerability, generating continuous risk scores based on prompt robustness metrics.

2.2 Data / Data Structure

Data flows through ARES starting from user input at the Blazor Frontend, normalized by the FastAPI Gateway, and processed by the Core Security Engines. Primary data sources include the 3,695 curated adversarial attack vectors ingested into the Qdrant Cloud Vector DB. Secondary data includes runtime telemetry, test runs, and findings stored in a relational SQLite/PostgreSQL database. Key data structures include Pydantic schemas for API contracts, such as the Category Contract Normalizer that maps 8 frontend categories (e.g., DirectPromptInjection, RoleManipulation) to 5 backend exploit classes (e.g., instruction_override, role_play_hijack).

2.3 SWOT Analysis

Strengths: ARES provides a highly sophisticated, closed-loop hardening mechanism that directly utilizes empirical breach traces to optimize prompts. The system features a highly accurate ML vector classifier and token-efficient syntax compression.

Weaknesses: The current implementation lacks production-grade OIDC authentication and authorization mechanisms. Furthermore, the system relies on external APIs (Groq, Gemini), making it susceptible to external rate limits and latency.

Opportunities: The platform can be expanded to support additional LLM providers and integrate seamlessly into existing DevSecOps pipelines. Future iterations could incorporate real-time, inline traffic interception and remediation.

Threats: Rapid evolution of adversarial prompt injection techniques may require frequent updates to the Qdrant vector corpus. The reliance on deterministic mock data for some UI components in the initial phase may obscure true operational metrics if not carefully managed.

2.4 Project Features

Feature: Live RAG Vector Retrieval
Purpose: To retrieve relevant attack vectors for red-teaming.
How It Works: Interrogates remote Qdrant Cloud populated with 3,695 vectors spanning JailbreakBench, HarmBench, and OWASP categories.

Feature: Continuous LLM Judge
Purpose: To evaluate target model responses for vulnerabilities.
How It Works: Uses models like Llama 3.3 to score responses, providing severity ratings and forensic explanations.

Feature: Closed-Loop Prompt Hardening
Purpose: To automatically secure vulnerable prompts.
How It Works: Accepts empirical test_id identifiers, extracts verified breach traces, and injects customized counter-defense rules into the prompt.

Feature: ML Vector Classifier
Purpose: To classify incoming vectors and retrain models.
How It Works: A Scikit-Learn linear classifier trained on Gemini embeddings, retrainable via an API endpoint.

2.5 User Classes and Characteristics

Security Analyst: Responsible for configuring and executing security tests. Requires deep understanding of prompt injection methodologies.

Prompt Engineer: Focuses on optimizing and hardening prompts utilizing the closed-loop feedback mechanism.

Incident Responder: Analyzes telemetry and finding reports to mitigate live threats.

Administrator: Manages system configurations, API keys, and orchestrates ML model retraining.

2.6 Design and Implementation Constraints

The system architecture mandates the use of .NET 8.0 for the Blazor Server frontend and Python 3.12+ for the FastAPI backend. External dependencies include the availability of Groq, Gemini, or NVIDIA API keys for LLM evaluations, and a Qdrant Cloud cluster for vector storage. The frontend and backend communicate strictly via typed JSON contracts over HTTP, with untrusted strings explicitly HTML-encoded by the Razor engine to prevent XSS vulnerabilities.

2.7 Design Diagrams

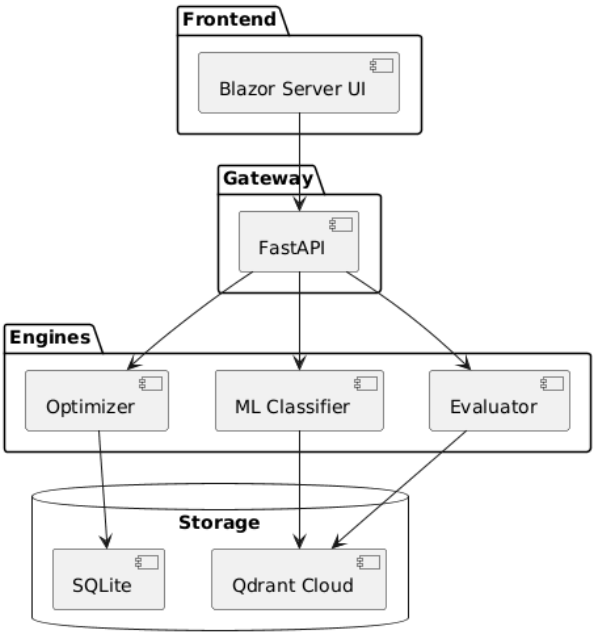


Figure 1: Architecture Diagram. Source: Project implementation and repository analysis.

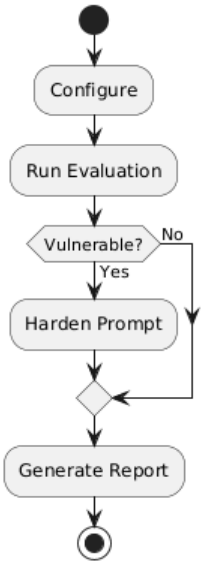


Figure 2: System Workflow Diagram. Source: Project implementation and repository analysis.

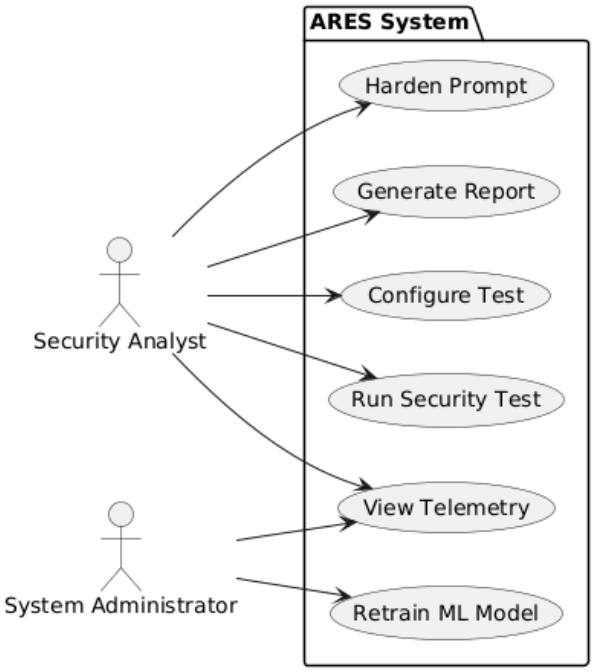


Figure 3: Use Case Diagram. Source: Project implementation and repository analysis.

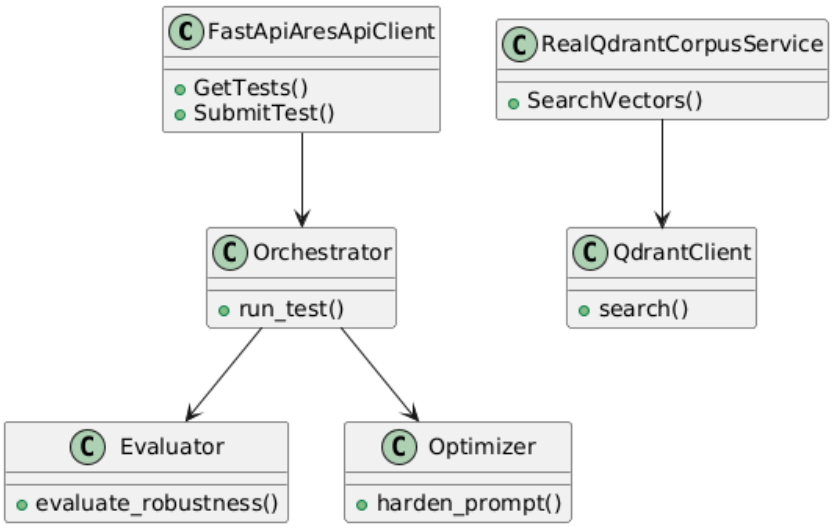


Figure 4: Class Diagram. Source: Project implementation and repository analysis.

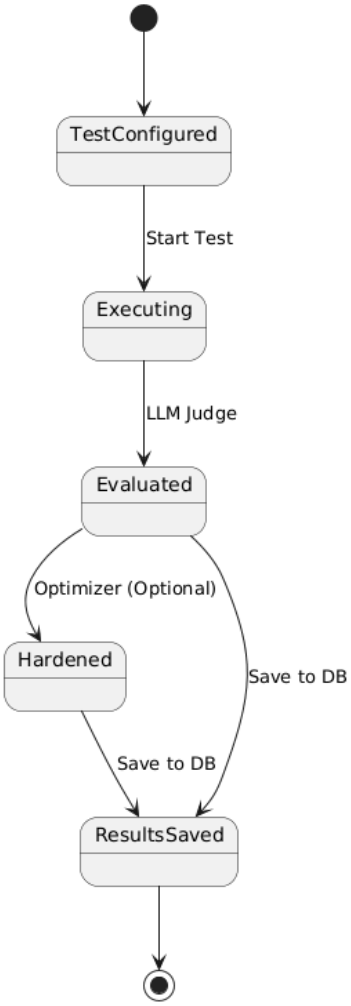


Figure 5: Activity Diagram. Source: Project implementation and repository analysis.

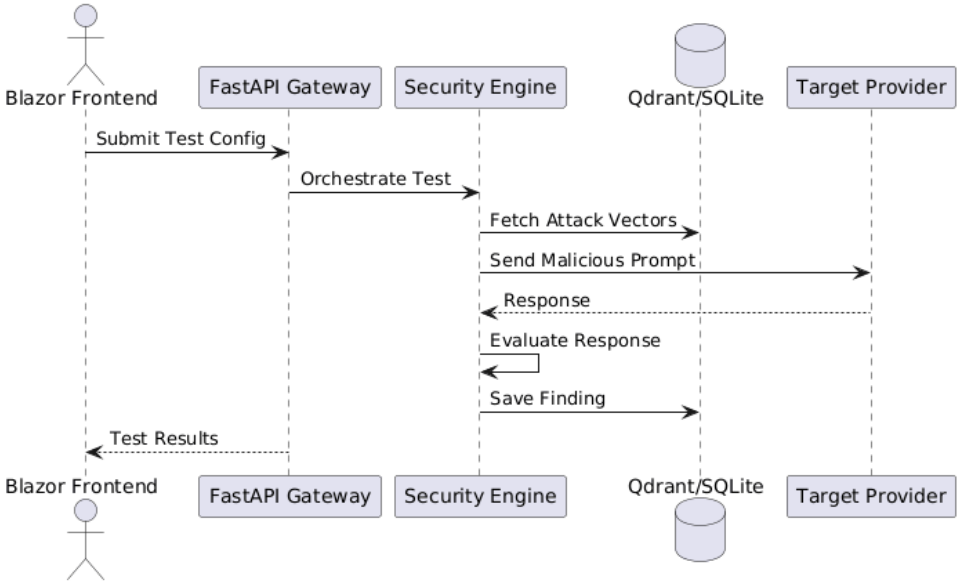


Figure 6: Sequence Diagram. Source: Project implementation and repository analysis.

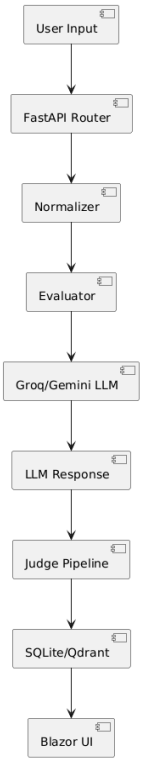


Figure 7: Data Flow Diagram. Source: Project implementation and repository analysis.

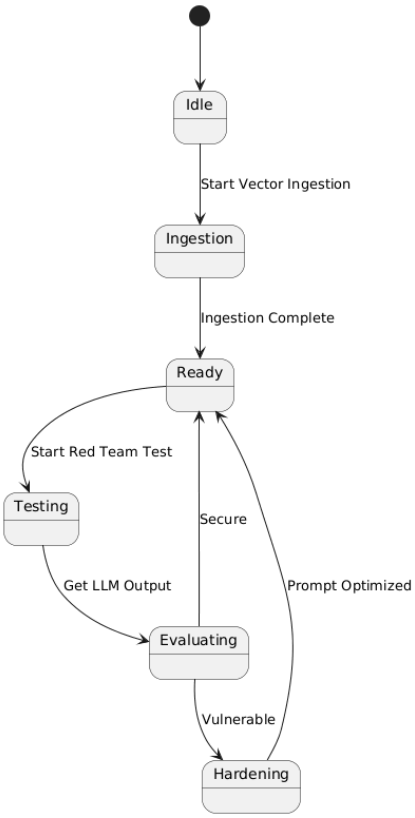


Figure 8: State Diagram. Source: Project implementation and repository analysis.

2.8 Assumption and Dependencies

Assumptions: It is assumed that the external LLM APIs (Groq, Gemini, NVIDIA) will remain accessible and maintain consistent response formats. It is also assumed that the initial deployment environment is isolated and does not require complex RBAC out of the box.

Dependencies: Python 3.12, .NET 8.0 SDK, FastAPI, Qdrant Cloud, SQLite/PostgreSQL, SQLAlchemy, Scikit-Learn, and various external LLM provider APIs.

3 SYSTEM REQUIREMENTS

3.1 User Interface

The user interface is built using Blazor Server (.NET 8) and comprises several key views:

- **Arena:** The primary interface for configuring and executing red-team sweeps and individual security challenges.
- **Overview/Dashboard:** Provides dynamic telemetry querying live SQLite/PostgreSQL records to render 14-day test, block, and incident trends.
- **Stats/ML Analytics:** Displays the performance of the ML Vector Classifier and provides controls for on-demand retraining.
- **Search:** Allows for semantic vector searches against the Qdrant Cloud corpus.
- **Reports:** Interface for generating and exporting security audit reports in Markdown, HTML, and JSON formats.

3.2 Software Interface

Table 1: Technology Stack

Software/Technology	Version	Purpose
.NET	8.0	Blazor Server Frontend
Python	3.12+	FastAPI Backend Core
FastAPI	0.115+	API Gateway and Routing
Qdrant	Cloud	Vector Database Storage
SQLite/PostgreSQL	Not specified	Relational Database for Telemetry

3.3 Database Interface

The system employs a dual-database architecture. A relational database (SQLite for local testing, PostgreSQL for production) is managed via SQLAlchemy to store TestRun, Finding, Evidence, User, and Project records. Concurrently, a remote Qdrant Cloud vector database stores and

indexes 768-dimensional Gemini embeddings for the 3,695 adversarial attack vectors. The FastApiAresApiClient and RealQdrantCorpusService act as the primary abstraction layers for database interactions.

3.4 Protocols

Communication between the Blazor frontend and the FastAPI backend occurs over standard HTTP/HTTPS protocols using RESTful JSON API contracts. Interaction with the external LLM providers and the Qdrant Cloud cluster also utilizes secure HTTPS. Internal service boundaries strictly adhere to typed Pydantic v2 schemas to ensure data integrity during transmission.

4 NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

The system is designed to handle live vector similarity analysis efficiently. The ML Vector Classifier is optimized to achieve 100% cross-validation accuracy across attack categories. The prompt optimizer employs token-efficient syntax compression to minimize inference overhead during target model execution. Specific quantitative throughput and response time benchmarks were not specified in the available repository.

4.2 Security Requirements

As a security-focused platform, ARES inherently implements rigorous evaluation protocols. Implemented features include:

- Untrusted strings are strictly HTML-encoded by the Razor engine to prevent XSS.
- Provider credentials are isolated within backend secret storage/environment variables.
- Prompt redaction occurs prior to durable logging and evidence export to protect sensitive business context.

Recommended future security implementations detailed in the architecture decisions include OIDC authentication, Role-Based Access Control (RBAC), and explicit authorization scoping by organization and application.

4.3 Software Quality Attributes

Maintainability: High, due to strict separation of concerns between the Blazor frontend, FastAPI gateway, and Core Security Engines.
Robustness: Ensured by comprehensive unit and integration test suites (99 Python tests, 14 .NET tests) evaluated in a CI/CD matrix workflow.
Adaptability: The system's modular architecture allows for the seamless integration of new LLM provider adapters and attack vectors.

5 OTHER REQUIREMENTS

Deployment requires Docker/Docker Compose capabilities as indicated by the presence of a docker-compose.yml file. The deployment environment must support the specific runtime dependencies for Python 3.12 and .NET 8.0. Continuous Integration is managed via GitHub Actions workflows validating changes across multiple Python versions and the .NET framework.

APPENDIX A: GLOSSARY

ARES: AI Resilience and Evaluation Security System.
LLM: Large Language Model.
RAG: Retrieval-Augmented Generation.
OIDC: OpenID Connect.
RBAC: Role-Based Access Control.
Qdrant: A highly performant vector search engine and database.

APPENDIX B: ANALYSIS MODEL

The analysis models are graphically represented in the Design Diagrams section (Section 2.7) of this document, illustrating the architecture, workflows, data flows, and state transitions of the ARES system.

APPENDIX C: ISSUES LIST

ID	Issue	Description	Impact	Status	Suggested Resolution
1	Missing Auth	No authentication or RBAC implemented in the initial release.	High	Open	Implement OIDC authentication and RBAC prior to production deployment.
2	External Dependency	System relies heavily on external LLM APIs (Groq, Gemini) for core functionality.	Medium	Open	Implement robust rate-limiting handling and fallback heuristic models.